Clustering Analysis

Prof. Dr. Matteo Marouf

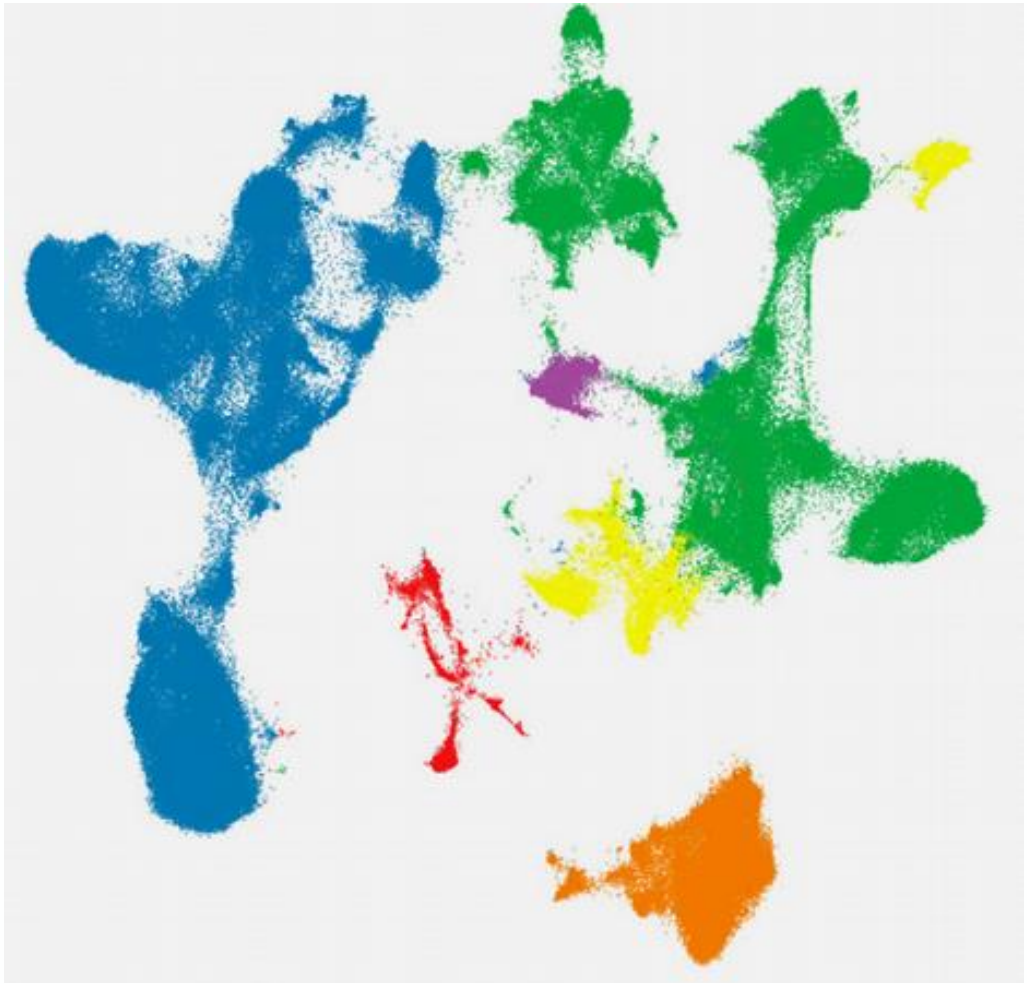
26.5. 2025

Agenda

- Supervised vs. Unsupervised Learning
- Clustering applications
- Clustering: K-means
- Determine the number of clusters
- Limitations of K-means

Unsupervised Learning

Clustering with K-means



Clustering is Unsupervised Learning aimed at understanding similarities

Training

The Data

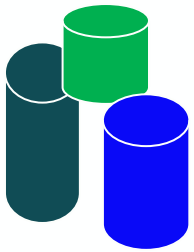
Learning data includes Information about the features of the objects like shape and color.

Unsupervised Training

The task is to learn similarities and differences between input examples to group them together.

Trained model

Deployment



Real-world Example

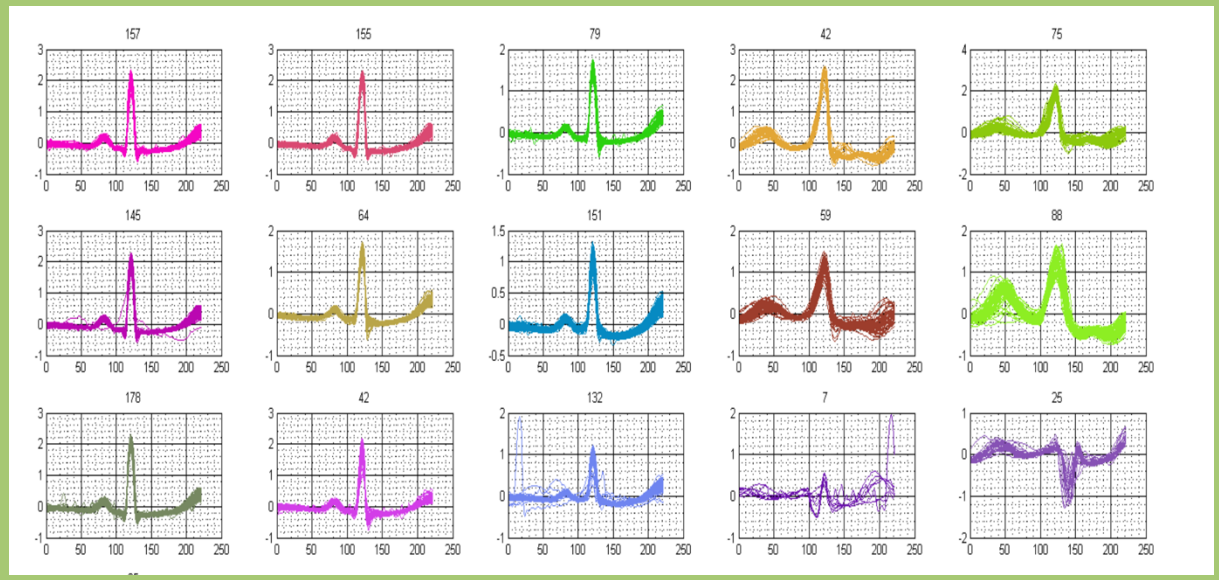
Heartbeat Morphology Identification by a Cardiologist

Context

A cardiologist analyzes a 7-day Holter ECG recording with hundreds of thousands of heartbeats.

Goal

Identify and group all possible heartbeat morphologies using clustering to support diagnosis and monitoring.



Input Feature

- Time-domain features (e.g., RR interval, QRS width)
- Morphological features (e.g., PCA on waveform shape)
- Possibly using deep embeddings (e.g., autoencoders)

Real-world Example

Customer Segmentation

🛍️ Context

A retail company wants to better understand **customer behavior** to improve **personalized marketing efforts**.

🎯 Goal

Use **clustering** to group customers into **segments** with similar characteristics and shopping behaviour.

📥 Input Features

- **Demographics**: Age, annual income
- **Behavior**: Spending score, browsing history, purchase history

Real-world Example

Social Listening

💬 Context

A brand wants to monitor and understand customer needs and opinions by analyzing social media comments and trends related to its products and services.

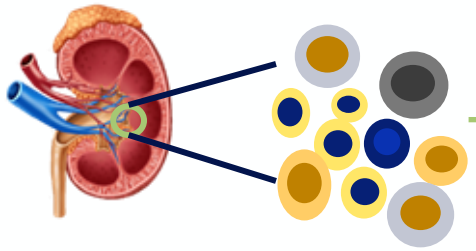
🎯 Goal

Use **clustering** to group social media posts into **topics or themes** (e.g., complaints, praise, feature requests, trends).

📥 Input Features

- **Textual features**: TF-IDF, word embeddings (e.g. Word2Vec)
- **Metadata (optional)**: Timestamp, platform, engagement..

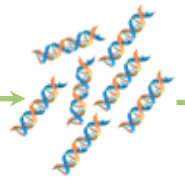
Clustering in practice
The Single cell RNA-seq pipeline



Disassociation

Single cell isolation

RNA



cDNA

Sequencing

	Gene 1	Gene 2
Cell 1	3	7
Cell 2	0	3
...	. . .	. . .
Cell N	19	6

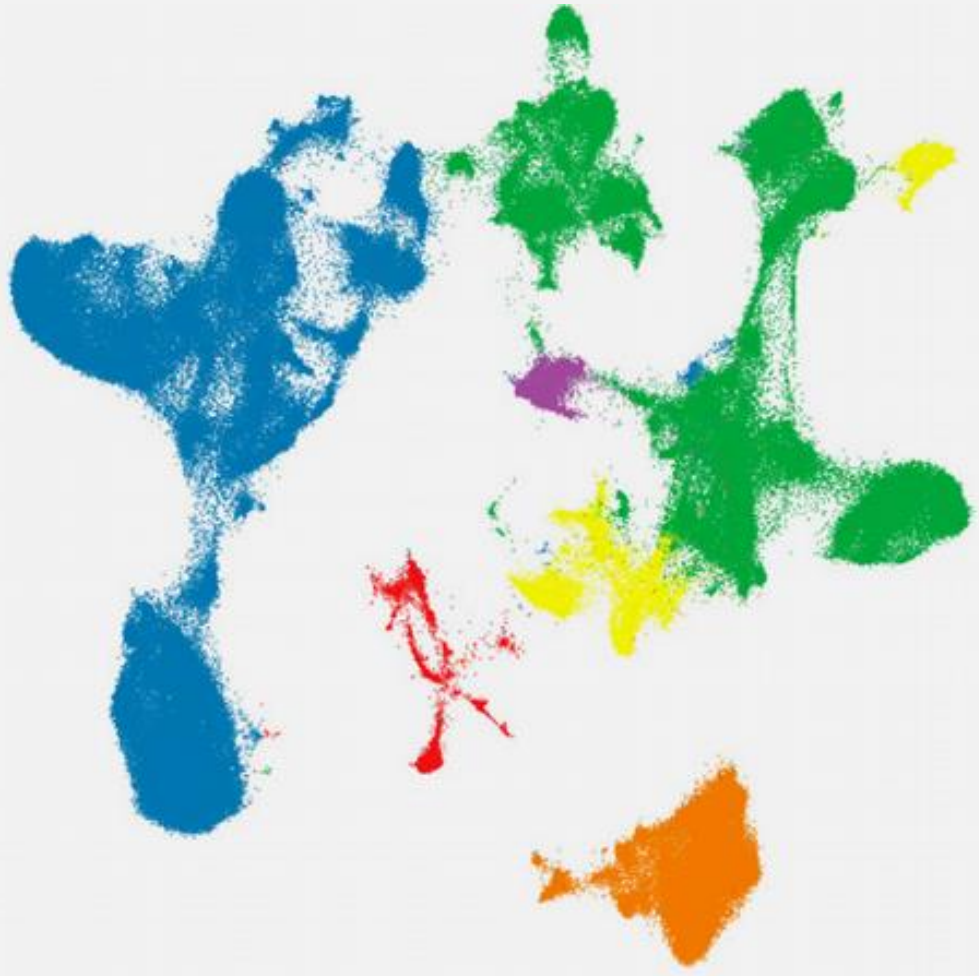
Cell - Genes matrix

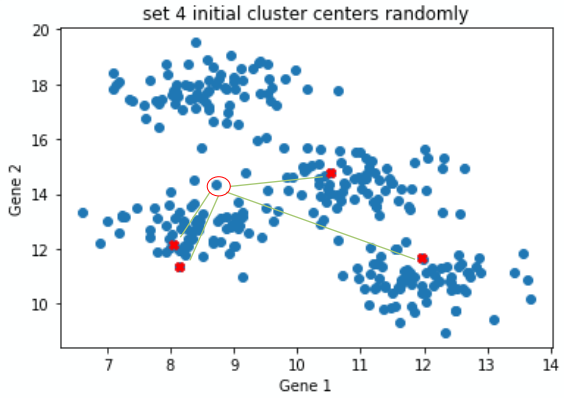
I run this everyday, and I need to find the different cell types in the tissue to understand their genes expression.

You need to cluster your data

Unsupervised Learning

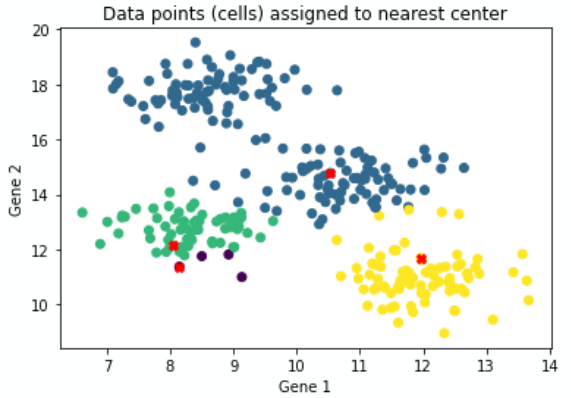
Clustering with K-means Algorithm

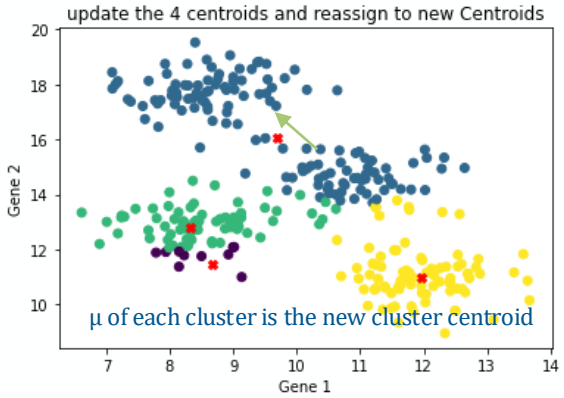


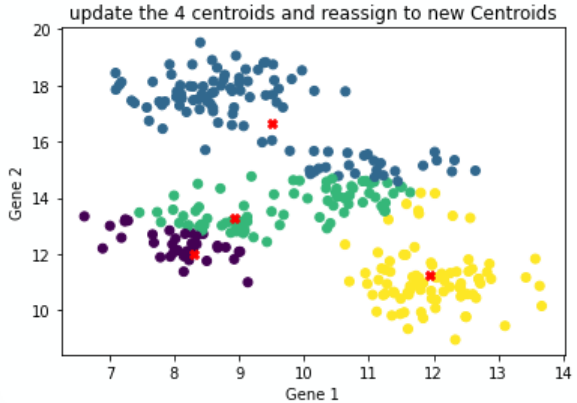


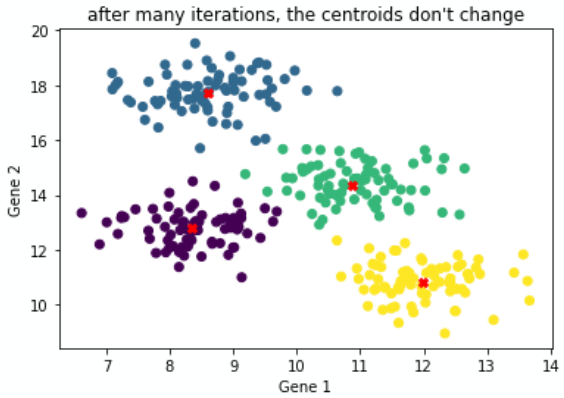
Quantify the Similarity between all data points and the centroids using squared euclidean distance:

$$||x-c||^2 = (x1 - c_1)^2 + (x2 - c_2)^2$$





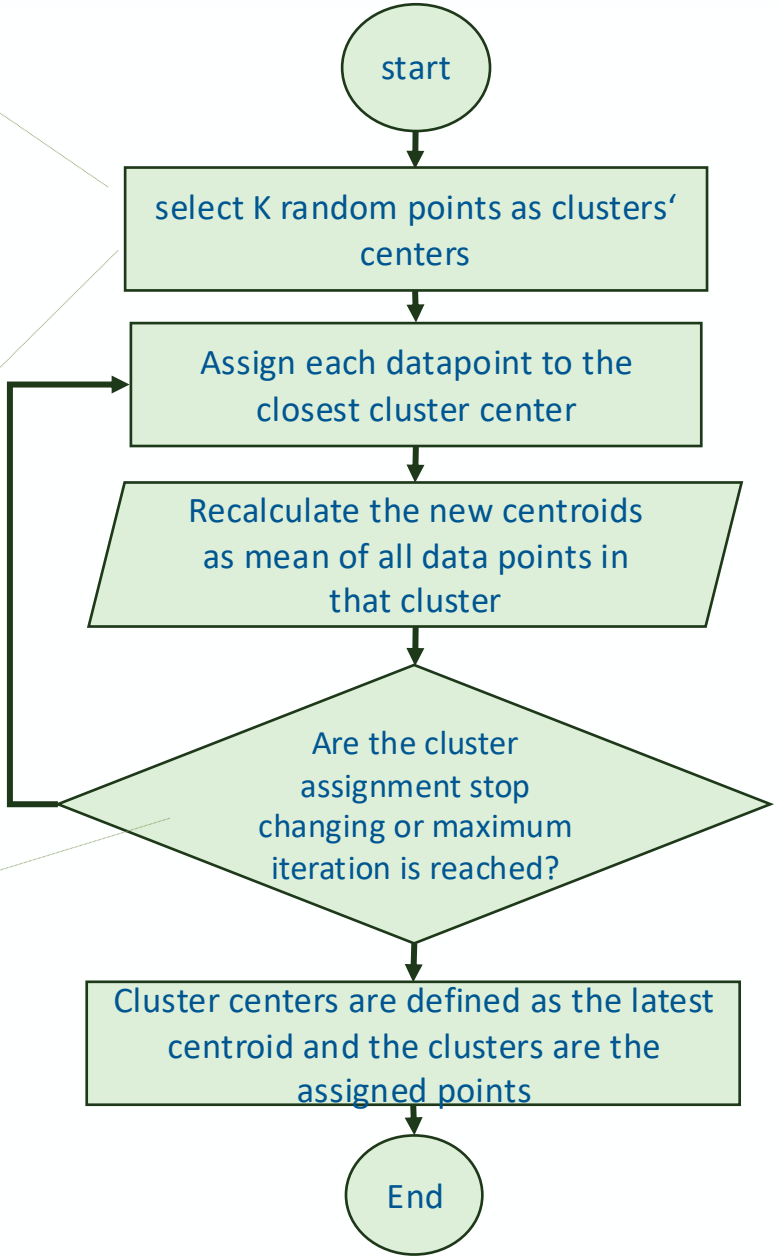




Number of clsuters is defined at the beginning

The initial points are selected randomly

Max. iterations is a predefined parameter

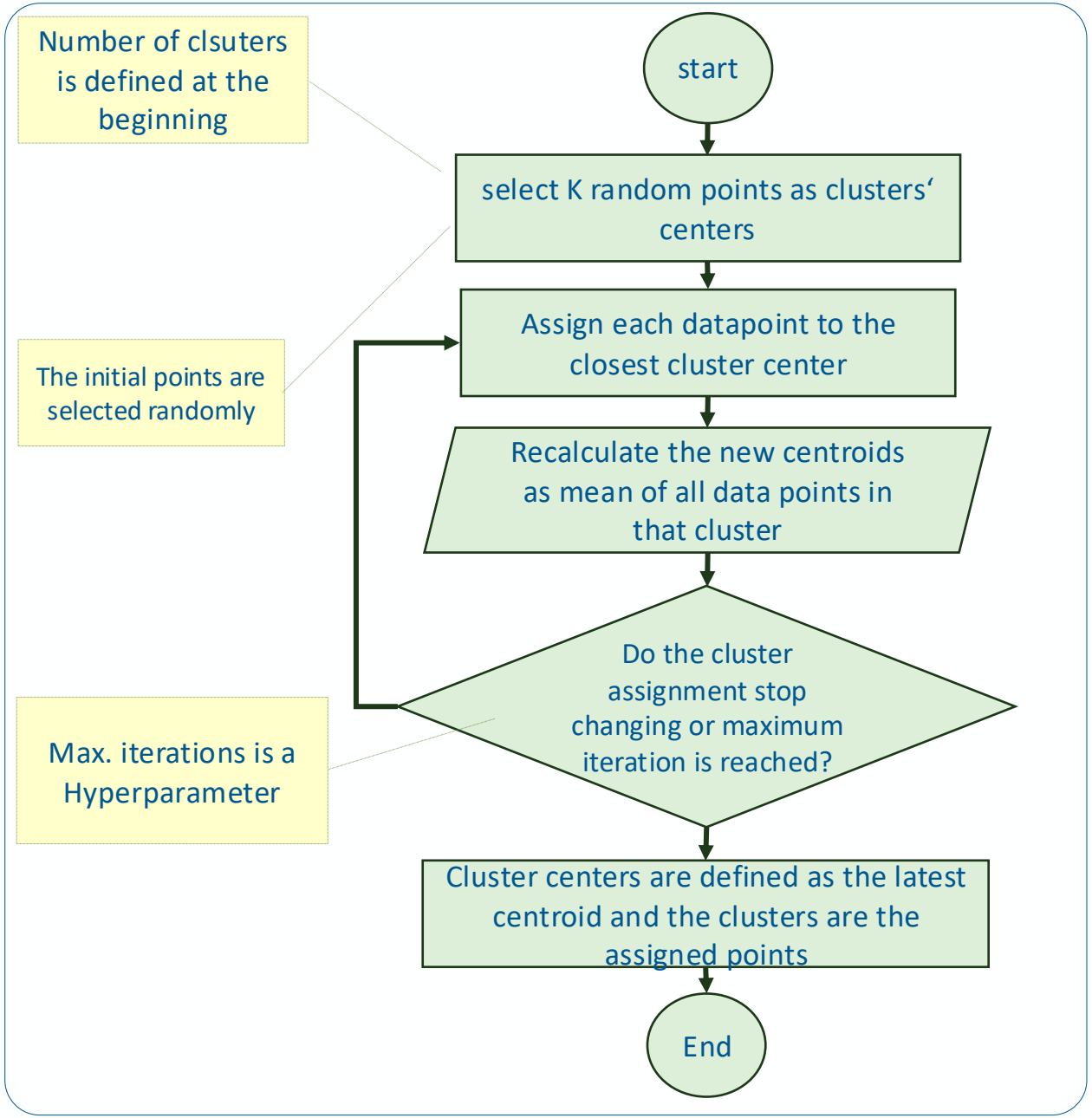


Vanilla K-means flowchart

Quiz

- **Is k-Means deterministic? Can K-means produce different results on different runs?**

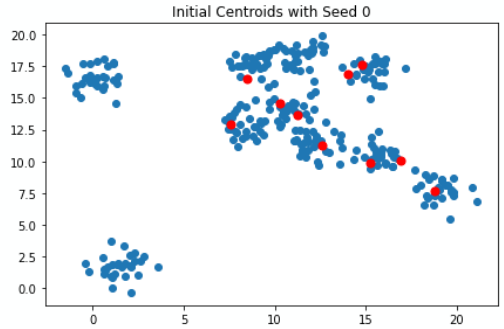
  - **A. of course,** K-means is always deterministic
  - **B. K-means may produce different clustering results across runs**
  - **C. It depends on the data we have**

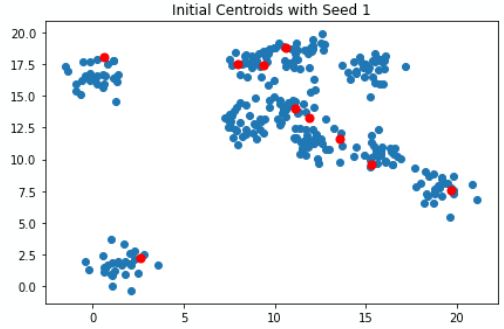


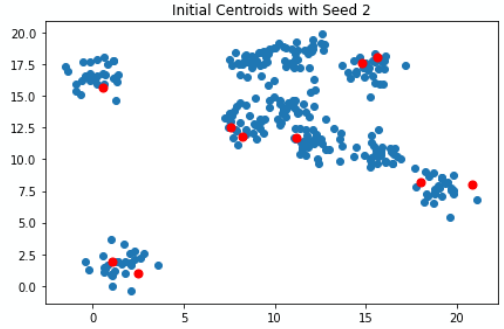
Vanilla K-means flowchart

Random selection of the centroid influences the outcome

- I tested the code but sometimes I run the code twice and I get two different outcomes for the clustering.

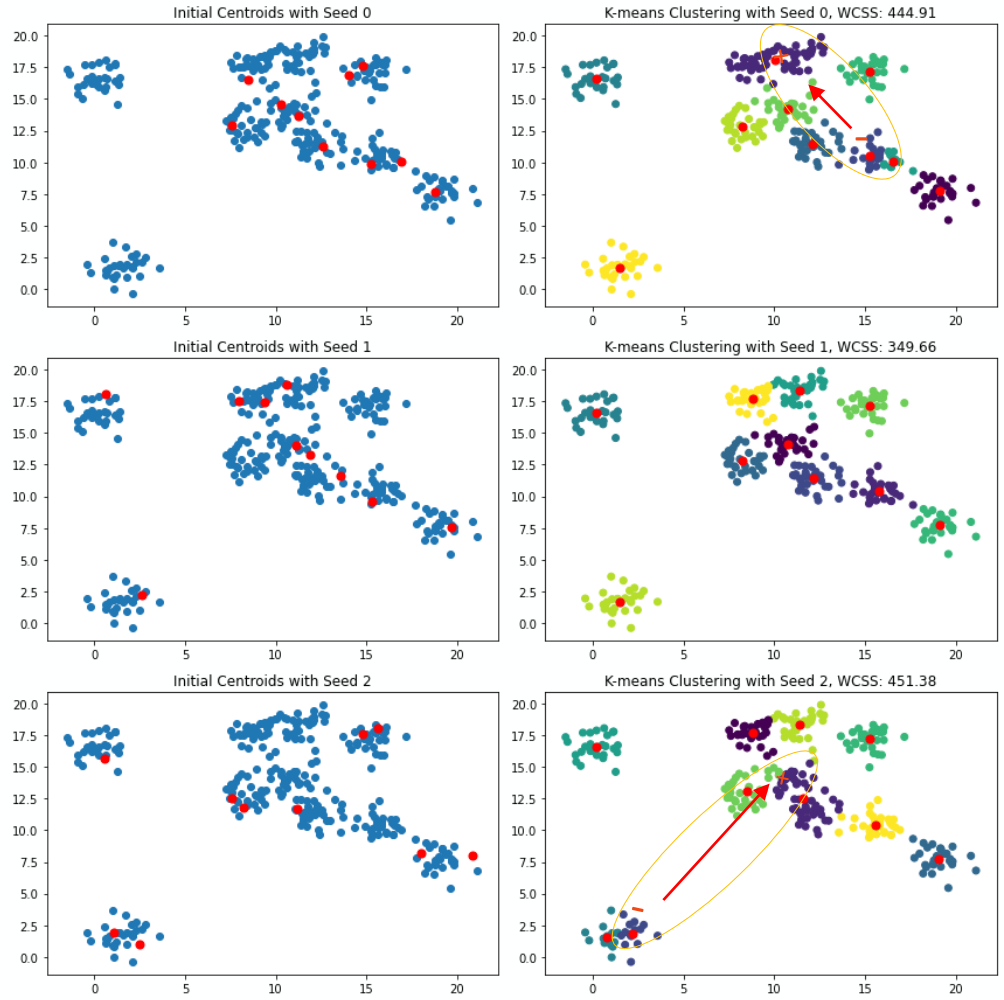






Random selection of the centroid influences the outcome

- I tested the code but sometimes I run the code twice and I get two different outcomes for the clustering.

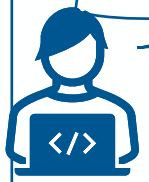


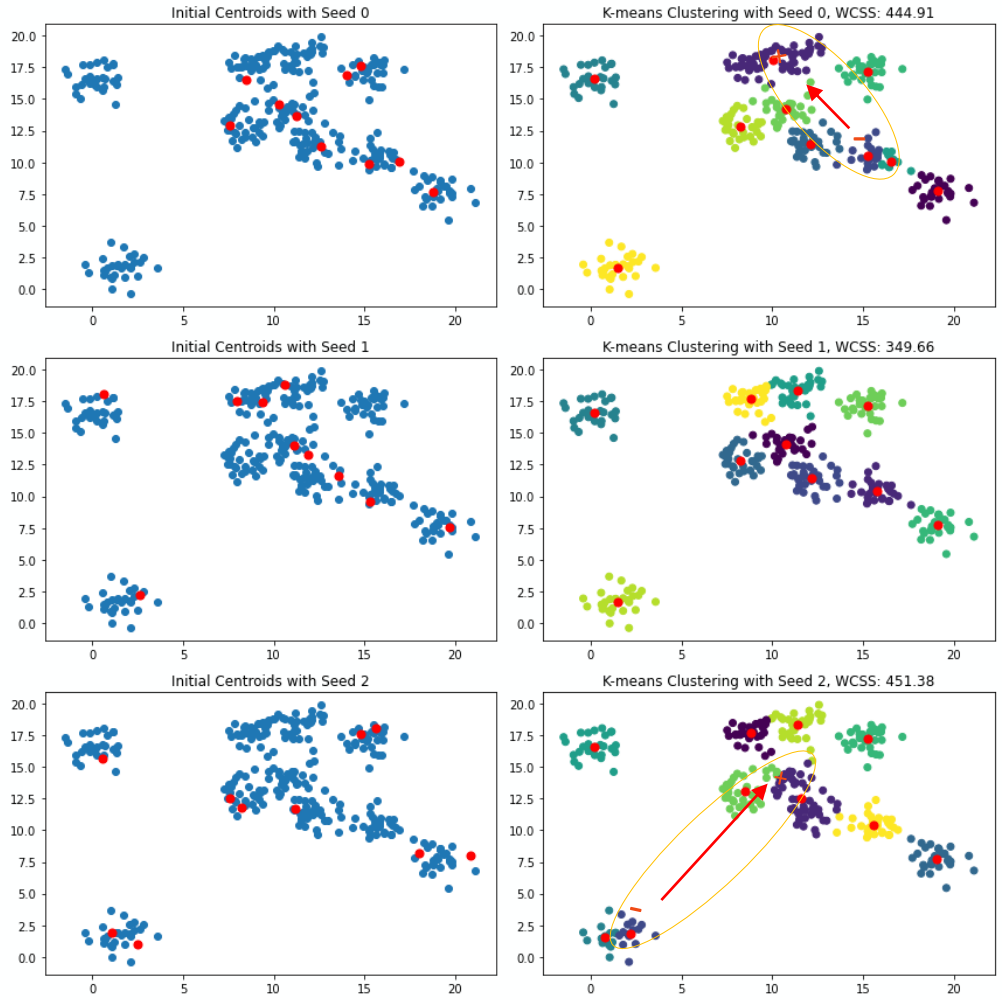
Here a minus sign (−) represents a centroid that is not needed, and a plus sign (+) a cluster where more centroids would be needed.

Random selection of the centroid influences the outcome

- I tested the code but sometimes I run the code twice and I get two different outcomes for the clustering.

- **K-means can get stuck in local optima**, as it's good at refining clusters locally but not relocating centroids globally.
  To improve results, it's helpful to **run the algorithm multiple times with different initial seeds** and choose the solution with the **most compact clusters.**





Here a minus sign (−) represents a centroid that is not needed, and a plus sign (+) a cluster where more centroids would be needed.

Solution 1: Cluster compactness to select good random initializations

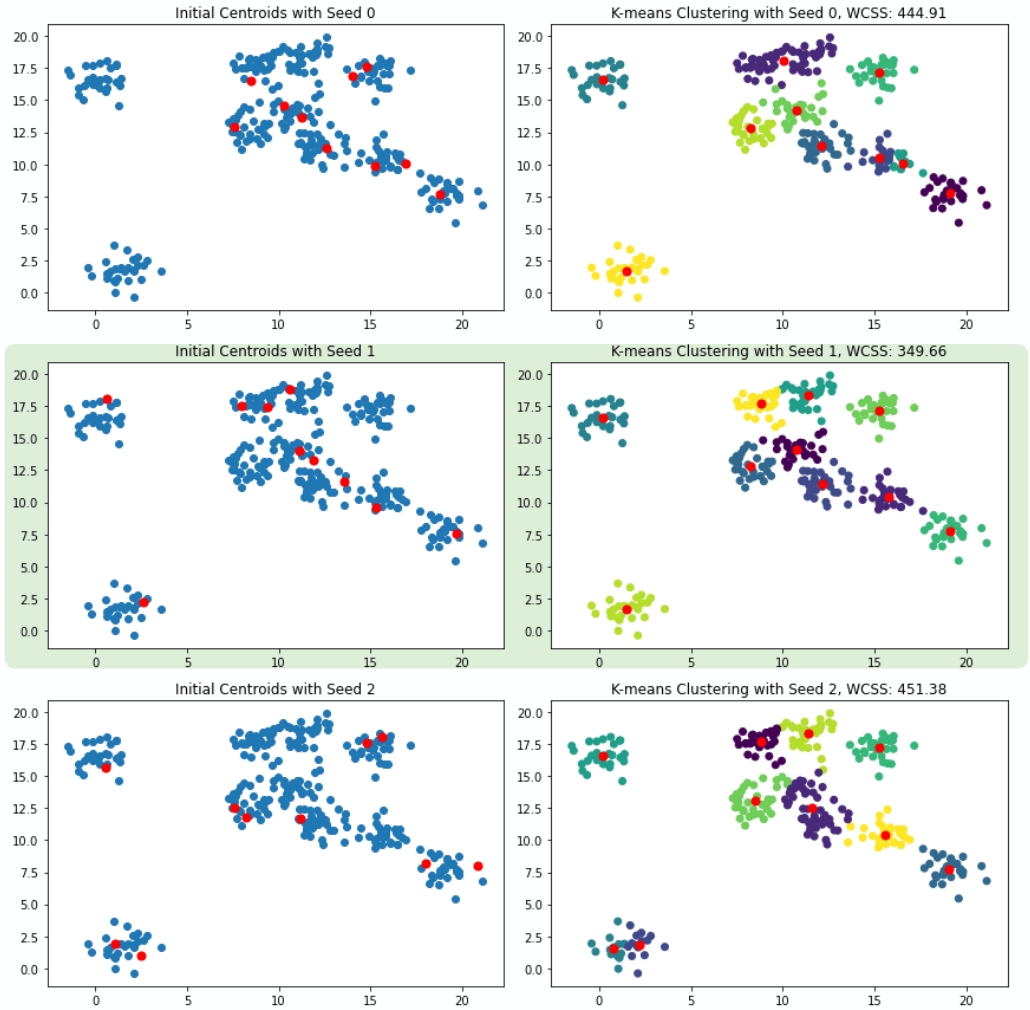
Cluster compactness:

1. Run the algorithm several times each with different centroid seeds.
2. Choose the best clustering output of the consecutive runs in terms of clusters' compactness.

To measure the cluster compactness we can use **Inertia or within-cluster sum-of-squares WCSS**:

$$W_k = \sum_{i=1}^{k} \sum_{x \in C_i} \|x - \mu_i\|^2$$

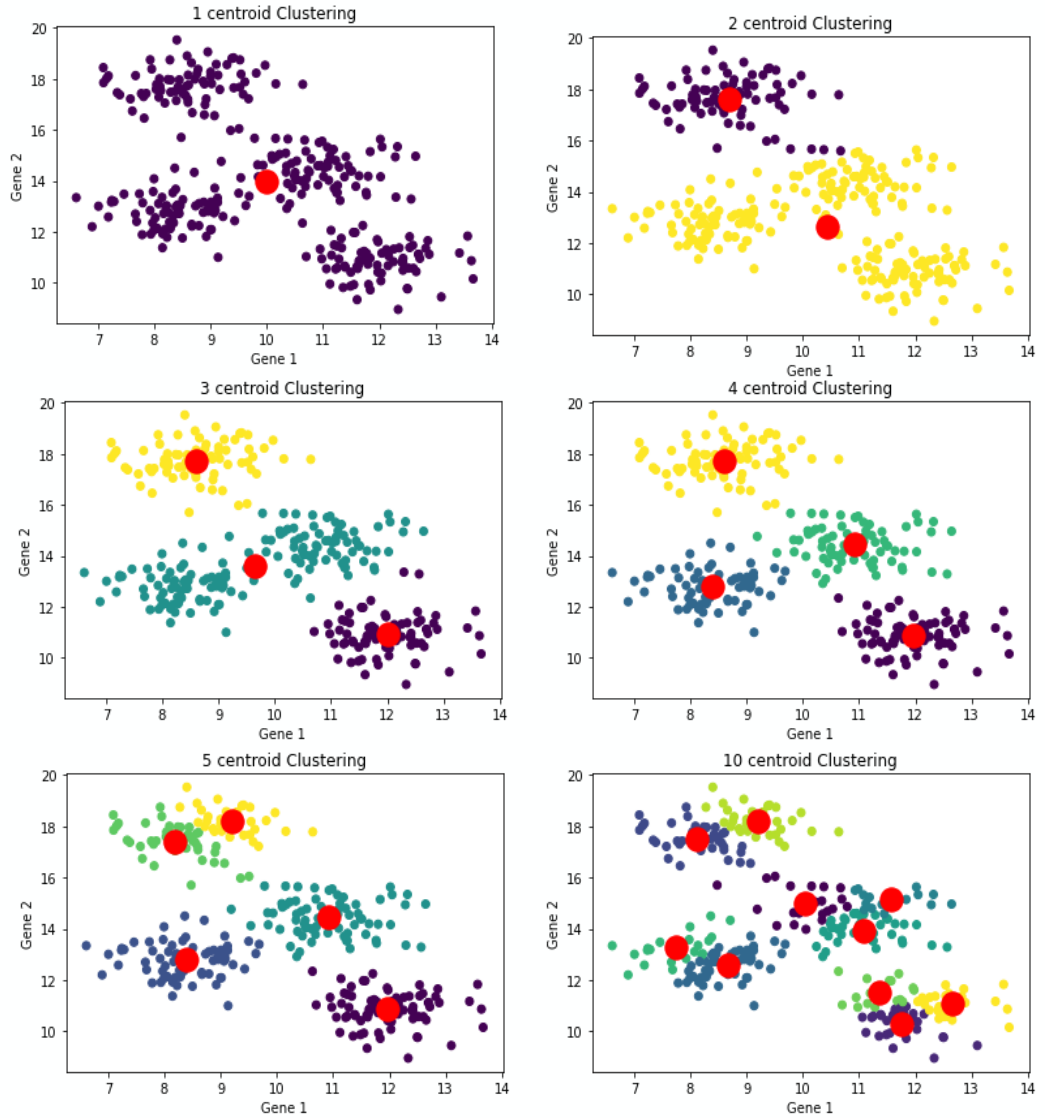
The K-means algorithm aims to choose centroids that minimize the inertia.



The choice of initial centroids can affect both the final clustering result and the speed at which the algorithm converges.

K-means requires predefined cluster number as an input

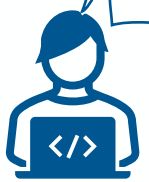
- I cannot always judge how many clusters (cell types) in the data, how to define the optimal cluster number?

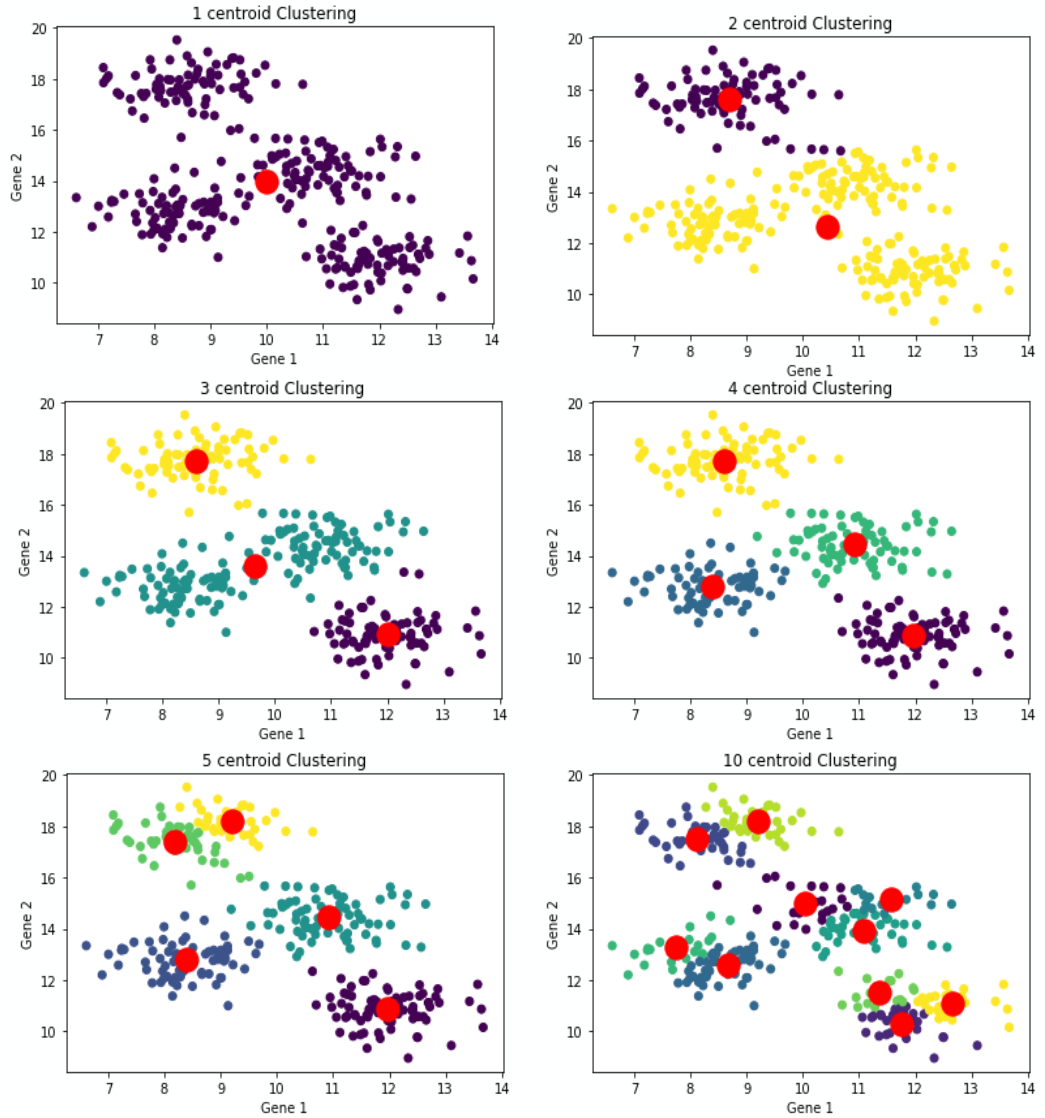


K-means requires predefined cluster number as an input

- I cannot always judge how many clusters (cell types) in the data, how to define the optimal cluster number?

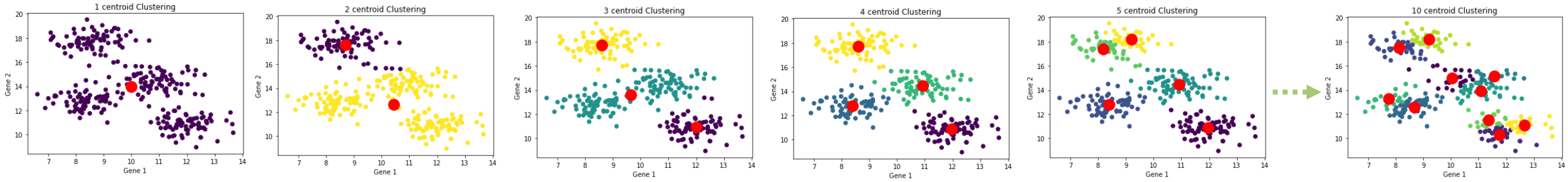
- I'll test **different numbers of clusters (e.g., 1–10)** to find the best fit.
- **Too few clusters** may miss important patterns, while **too many** can lead to overfitting.
- The key challenge is knowing **when to stop** and **how to choose the optimal number of clusters?**





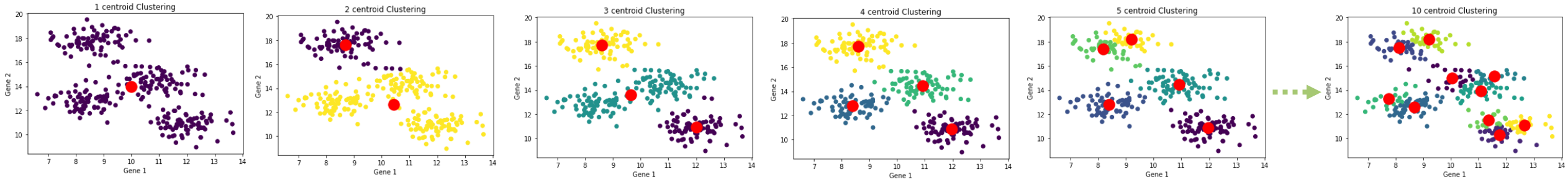
The elbow method can help find the number of clusters

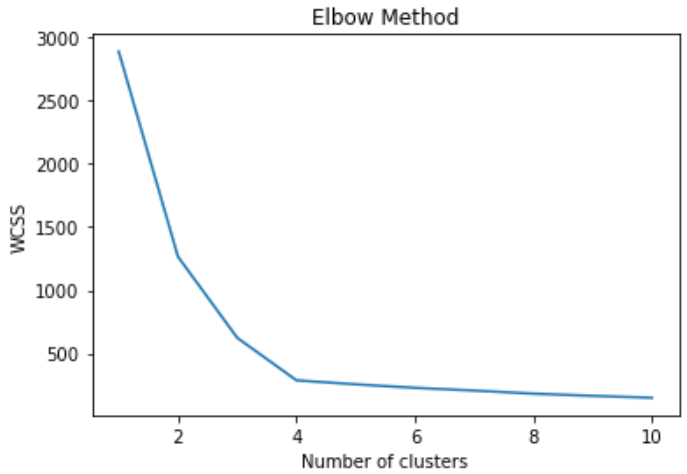
For each cluster number, calculate the total within cluster sum of squares (inertia) and sum it across all clusters



The elbow method can help find the number of clusters

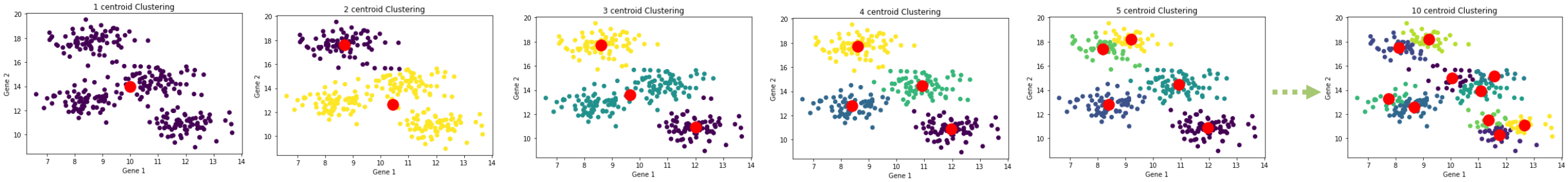
For each cluster number, calculate the total within cluster sum of squares (inertia) and sum it across all clusters

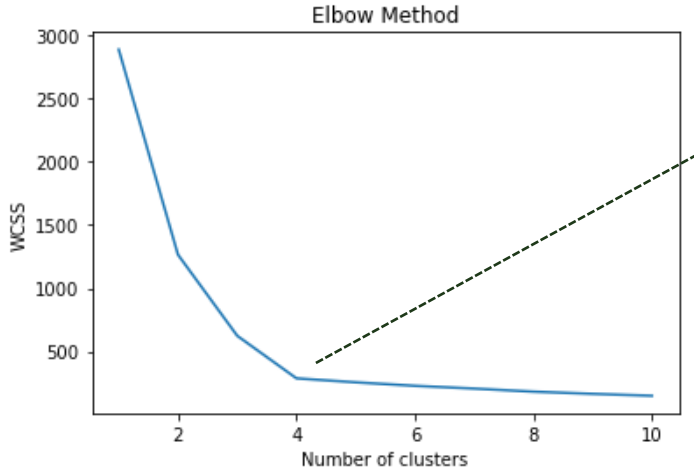




The elbow method can help find the number of clusters

For each cluster number, calculate the total within cluster sum of squares (inertia) and sum it across all clusters

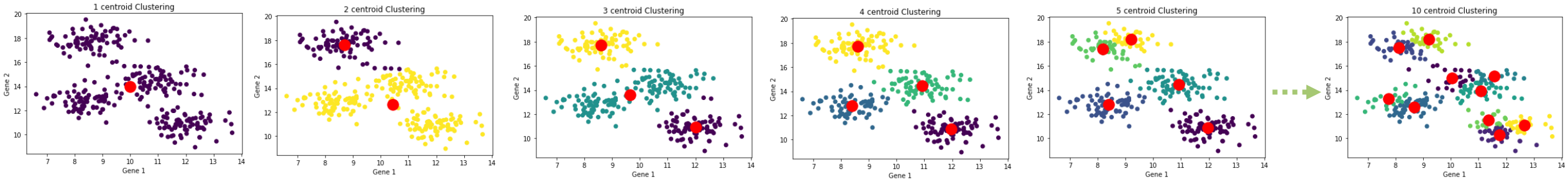


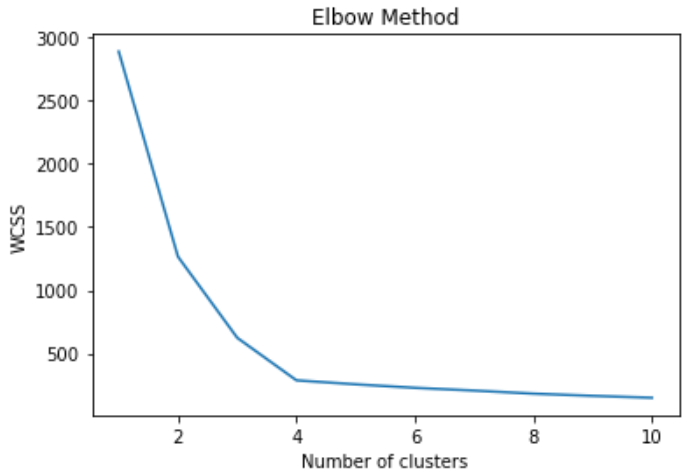


- Beyond the elbow point, the inertia decreases slowly, indicating that the additional clusters are not adding significant value.
- Hence, the elbow point is often a good **trade-off** between increasing complexity (more clusters) and sufficient data modeling.
- This is heuristic and subjective since the selection of the elbow point can be difficult.

The elbow method can help find the number of clusters

For each cluster number, calculate the total within cluster sum of squares (inertia) and sum it across all clusters





Bottomline: The elbow method

- Calculate inertia for different repeats
- select the "elbow" point because it represents the point at which adding more clusters doesn't provide much better modeling of the data.
- It is not the optimal solution

Homework: read through the paper to understand why there is no optimal solution in cluster analysis and to learn about different methods to choose the number of clusters

Limitations

- **Subjectivity:**
  The choice of the "elbow point" can be subjective and might vary between individuals analyzing the same data.

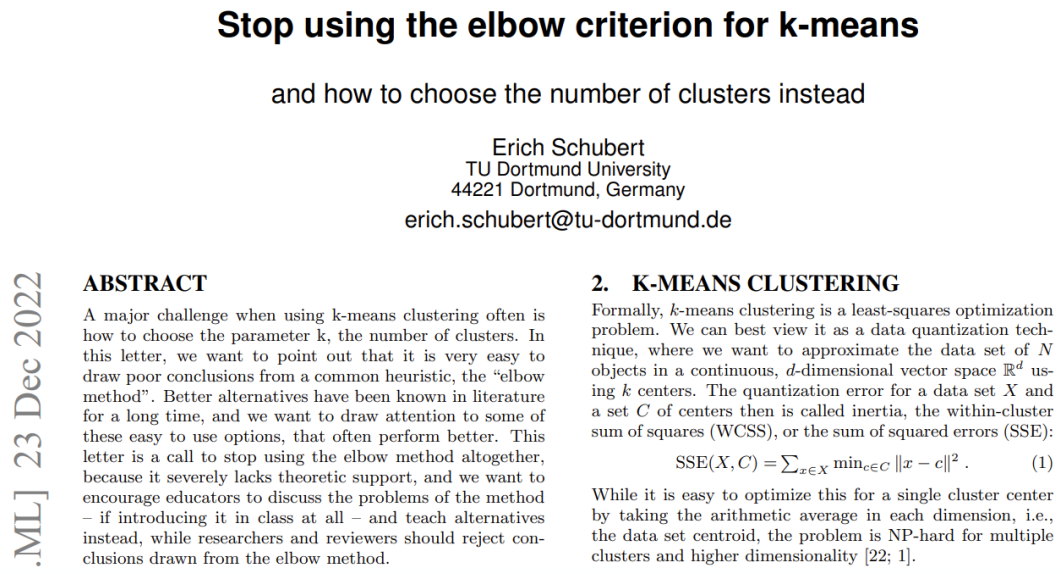
- **Unsuitable for All Distributions:**
  It assumes that clusters are spherical and equally sized, which may not hold for complex datasets with irregularly shaped or differently sized clusters.

- **Sensitivity to Initialization:**
  K-means itself is sensitive to initial cluster centroids, which can affect the WCSS values and, consequently, the choice of the optimal K.

- **İnefficient for Large Datasets:**
  For large datasets, calculating WCSS for a range of K values can be computationally expensive and time-consuming.

- **Limited to K-means:**
  It specifically applies to K-means clustering and may not be suitable for other clustering algorithms with different objectives.

Homework



Stop using the elbow criterion for k-means

and how to choose the number of clusters instead

Erich Schubert
TU Dortmund University
44221 Dortmund, Germany
erich.schubert@tu-dortmund.de

ABSTRACT

A major challenge when using k-means clustering often is how to choose the parameter k, the number of clusters. In this letter, we want to point out that it is very easy to draw poor conclusions from a common heuristic, the "elbow method". Better alternatives have been known in literature for a long time, and we want to draw attention to some of these easy to use options, that often perform better. This letter is a call to stop using the elbow method altogether, because it severely lacks theoretic support, and we want to encourage educators to discuss the problems of the method – if introducing it in class at all – and teach alternatives instead, while researchers and reviewers should reject conclusions drawn from the elbow method.

2. K-MEANS CLUSTERING

Formally, k-means clustering is a least-squares optimization problem. We can best view it as a data quantization technique, where we want to approximate the data set of N objects in a continuous, d-dimensional vector space $\mathbb{R}^d$ using k centers. The quantization error for a data set X and a set C of centers then is called inertia, the within-cluster sum of squares (WCSS), or the sum of squared errors (SSE):

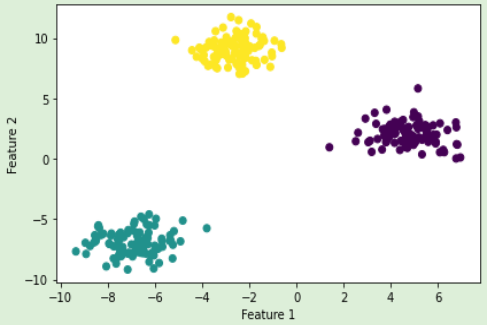
$$\mathrm{SSE}(X, C) = \sum_{x \in X} \min_{c \in C} \|x - c\|^2 . \quad (1)$$

While it is easy to optimize this for a single cluster center by taking the arithmetic average in each dimension, i.e., the data set centroid, the problem is NP-hard for multiple clusters and higher dimensionality [22; 1].

"There is no "optimal" solution in cluster analysis, but it is an explorative approach that may yield multiple interesting solutions, and interestingness necessarily is a subjective decision of the user."

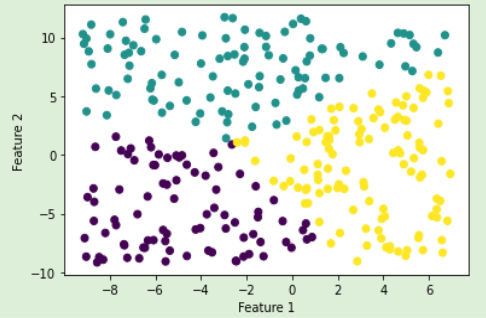
Gap Statistic to determine the optimal number of clusters

1



How good is the model performing on the data?
Calculate the $logW(K)$ where W is the within cluster dispersion or sum of squared distances

2

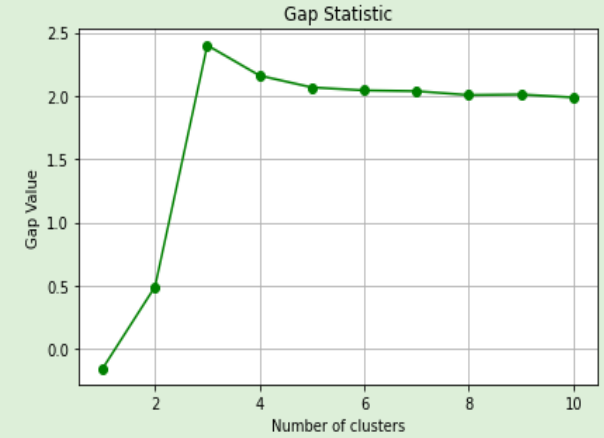


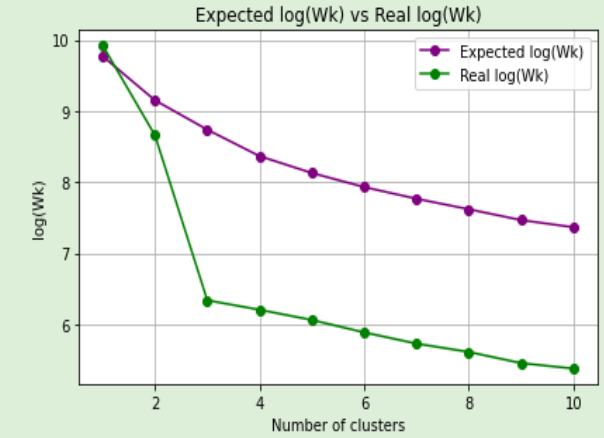
How good is it performing for simulated reference data uniformly distributed over a rectangular containing the data?
Calculate the $logW_{unif}(K)$ and average over multiple simulations B

3

Repeat and calculate the Gap(K) for different Ks:

$$Gap(K) = \frac{1}{B}\sum_{b=1}^{B} logW_{unif}(K) - logW(K)$$





4

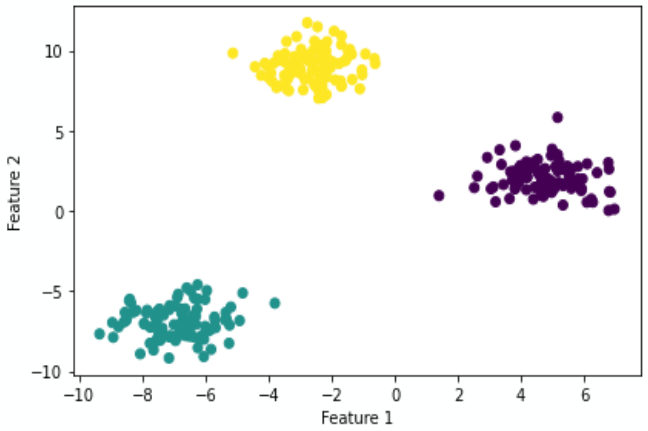
The optimal number of clusters K is the smallest k such that:

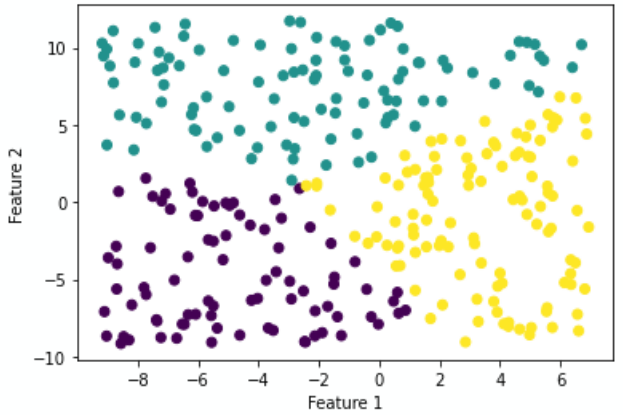
Gap(k) >= Gap(k+1) - SE(k+1)

5

Standard error is needed to provide a measure of uncertainty for the comparison with the simulated data

Gap Statistic to determine the optimal number of clusters





Intuition Behind the Gap Statistic

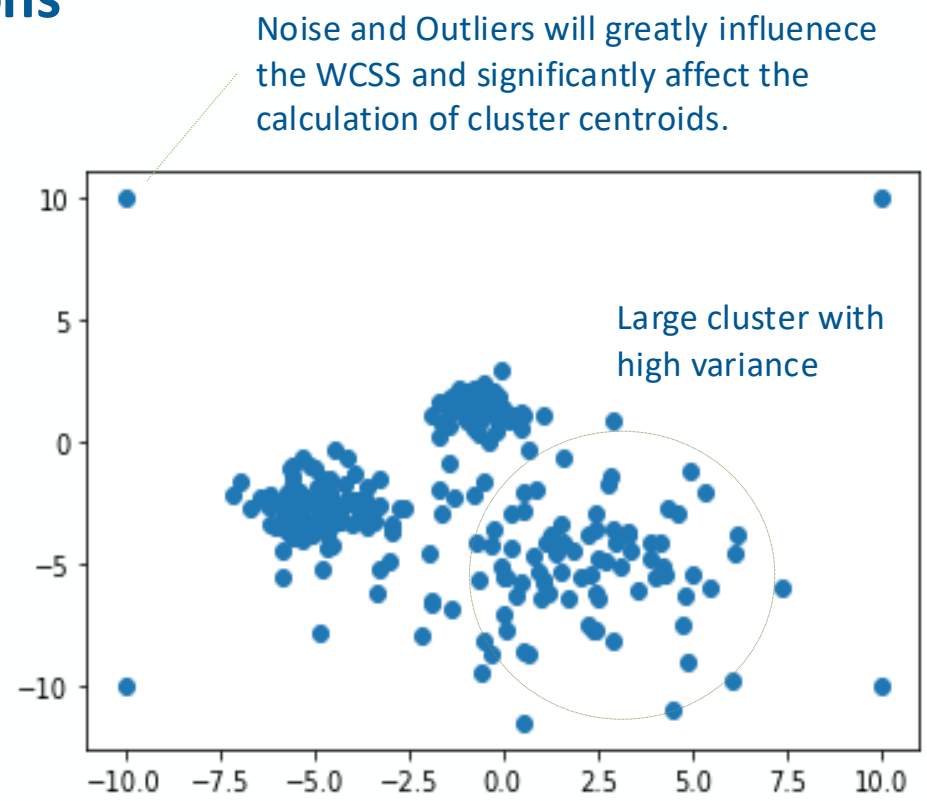
The Gap Statistic essentially tells us how far away our data clustering is from what we'd expect by chance.

A higher gap suggests that our clusters are well-separated and not a result of random variation. So, by maximizing this "gap," we can find a clustering configuration that captures the natural structure in the data.

K-means Pros & Cons

Main Limitations

- **Sensitive to outliers**: Outliers can distort cluster centroids.
- **Local optima**: May converge to suboptimal solutions.
- **Scaling matters**: Assumes features are on similar scales; otherwise, results are skewed (can be worked around)
- **Not suited for nominal data**: Uses squared Euclidean distance, which isn't meaningful for categorical features.
- **Assumes similar cluster sizes**: May not perform well if clusters vary in size or compactness.

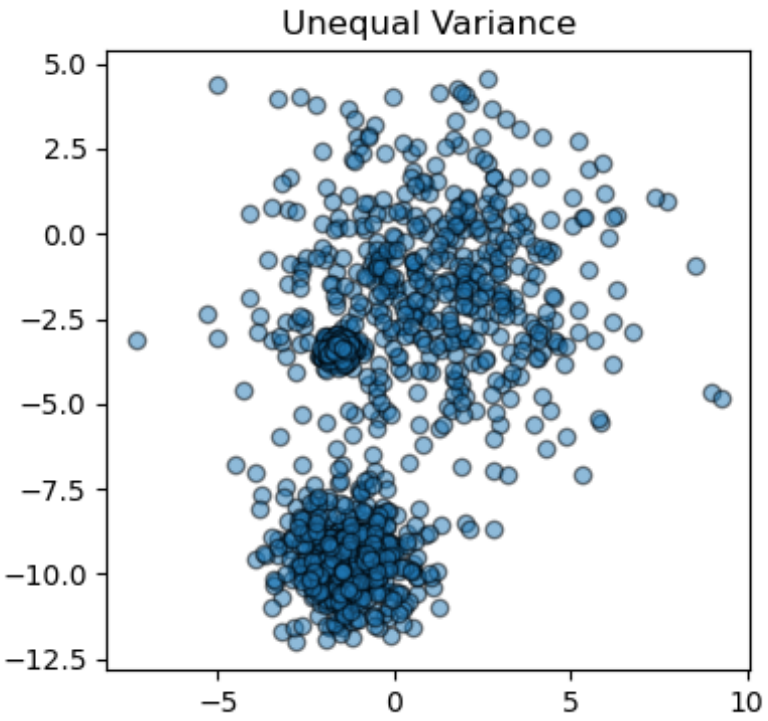


Example on how outliers and different cluster size can influence the WCSS calculation leading to misleading results.

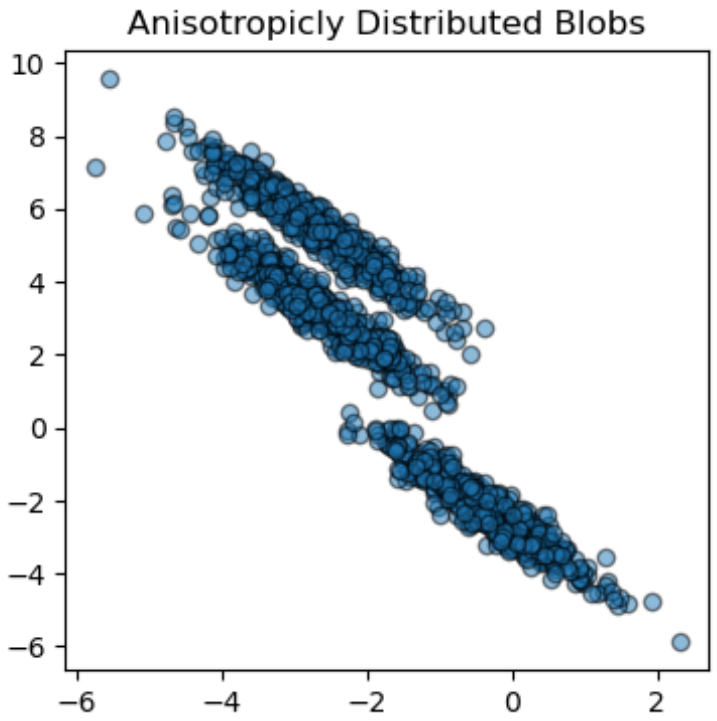
Bottom Line:
K-means clustering is a simple and efficient algorithm that is effective in situations when clusters are spherical and of similar size, making it a good choice for quick exploratory data analysis.

Quiz

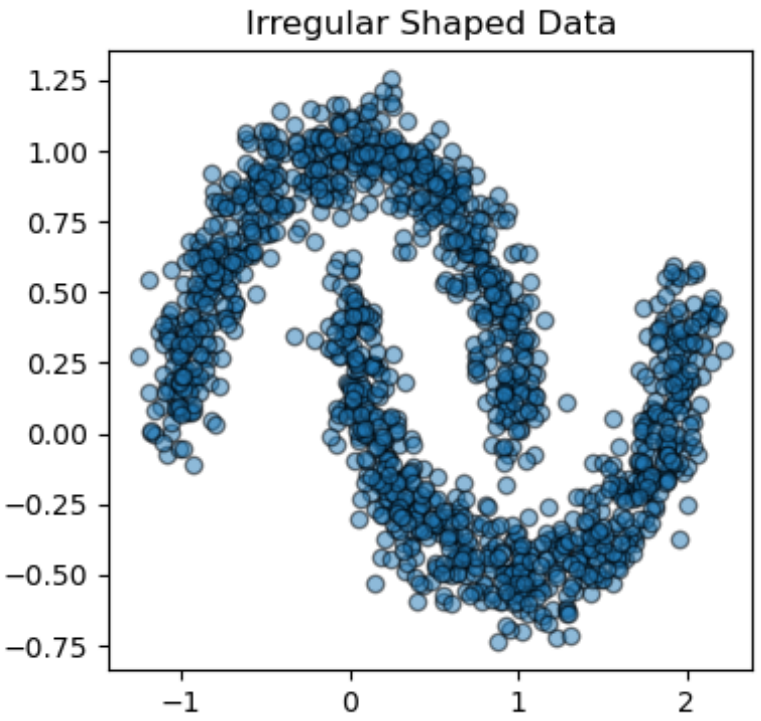
Where would K-means perform at best to cluster these points?



A



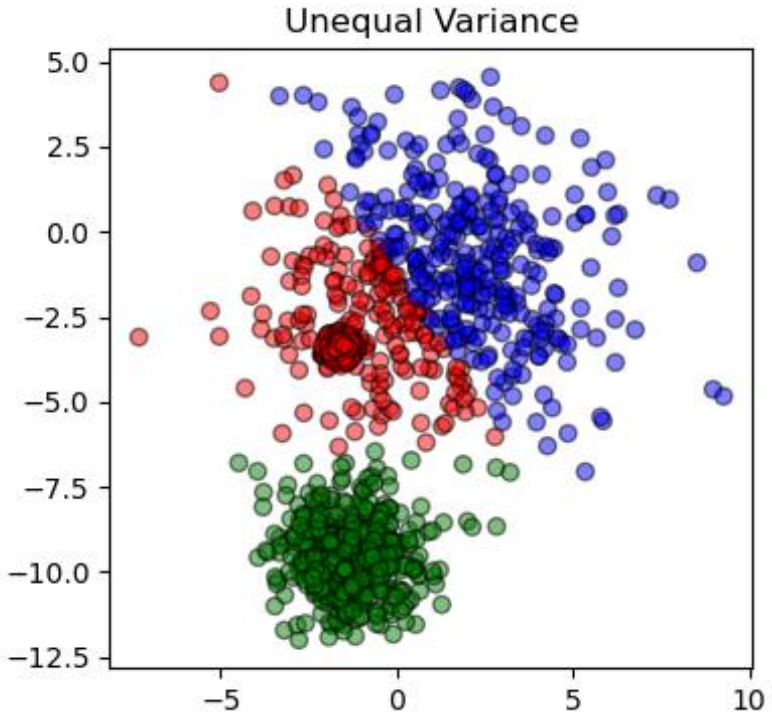
B



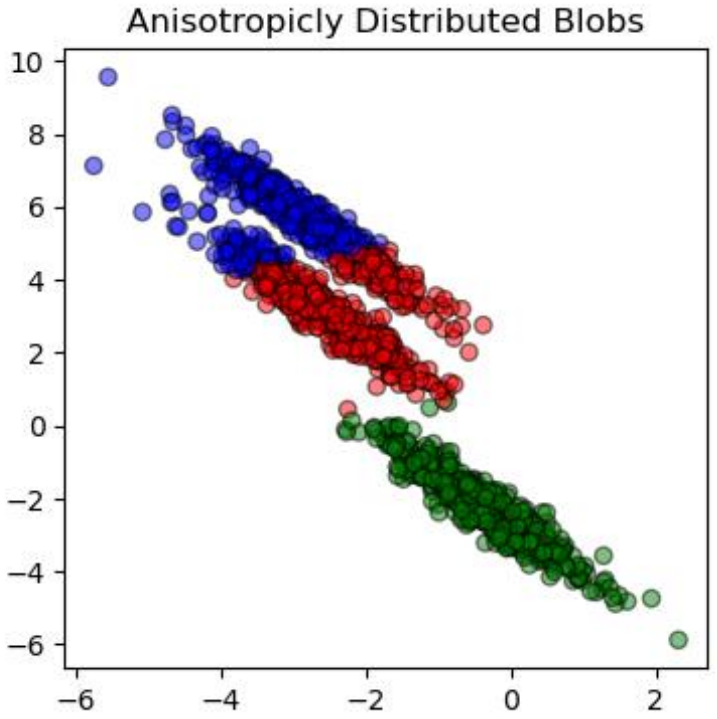
C

Quiz

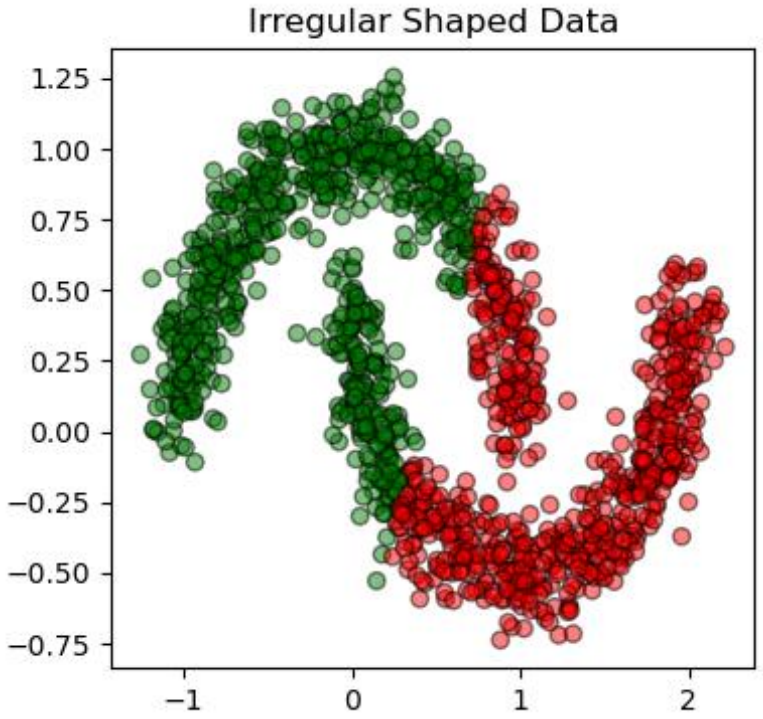
Where would K-means perform at best to cluster these points?



A



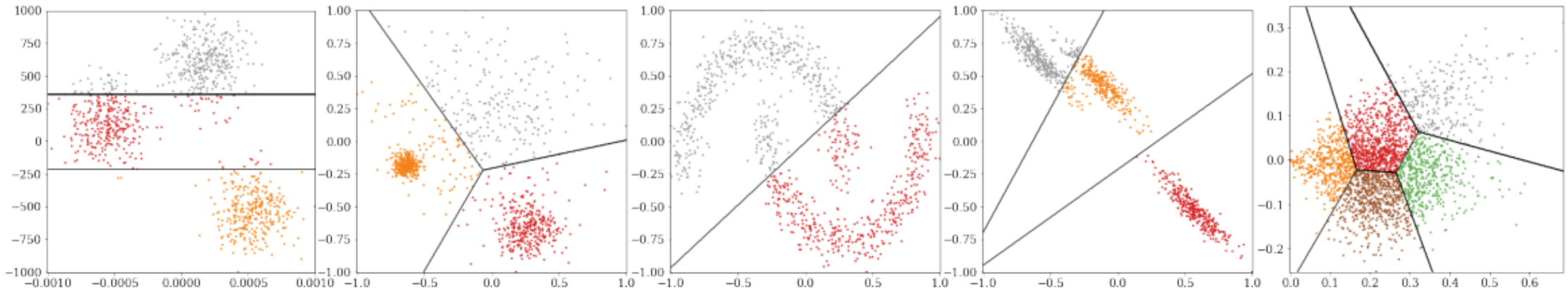
B



C

K-Means struggles with non-Gaussian shaped data, often producing misleading clusters. Despite these pitfalls, it remains a valuable tool for data inspection when its limitations are acknowledged and managed.

Homework
further examples when K-means fails to provide a good solution



Source, stop using the elbow criterion for K-means, Schubert, 2022.

K-means clustering (Python)
Toy-example code

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

clf = DecisionTreeClassifier(criterion = "entropy", min_samples_leaf = 3)
# Lots of parameters: criterion = "gini" / "entropy";
#                     max_depth;
#                     min_impurity_split;

clf.fit(X, y) # It can only handle numerical attributes!
# Categorical attributes need to be encoded, see LabelEncoder and OneHotEncoder

clf.predict([x]) # Predict class for x

clf.feature_importances_ # Importance of each feature
clf.tree_ # The underlying tree object

clf = RandomForestClassifier(n_estimators = 20) # Random Forest with 20 trees
```

Limitation of K-means on big-data

Computational Complexity: The algorithm needs to iterate through a large number of data points multiple times, which can be computationally expensive and time-consuming.

Memory Limitations: Storing big data in memory can be a challenge. This is especially true when the data is distributed across multiple machines in a cluster.

Initialization Sensitivity: K-means is sensitive to the initial placement of centroids. With big data, finding a good initial placement can be more challenging.

Scalability: Traditional K-means may not scale well with the increase in data size. This has led to the development of variations like Mini Batch K-means and K-means++ which are more efficient on larger datasets.

High Dimensionality: Big data often comes with high dimensionality, which can make the clustering process more complex and the results harder to interpret.

Pyspark and RAPID can help with that

RAPIDS

APACHE PySpark

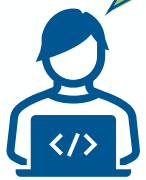
Bonus slide: More evaluation metrics for clustering

Many metrics has been developed for evaluating clustering. However, each rely on some data-driven assumption.

1. **Silhouette Coefficient:** A normalized measure of how similar an object is to its own cluster compared to other clusters.
2. **Completeness Score:** good clustering algorithm should assign all samples with the same true label to the same cluster.
3. **Normalized Mutual Information (NMI):** indicates better alignment between clusters and true labels.
4. **Davies-Bouldin Index:** measures within-cluster similarity to dissimilarity ration.
5. **Others: Calinski-Harabasz Index, Rand Index, Adjusted Rand Index (ARI), Homogeneity Score**

So, what to choose? And how to report it? And how to vizualize it?

Remember: What you use technically to achieve the best clustering is not necessarily the most suitable and interesting metric to report



Clustering is exploratory. It's about understanding structure, not predicting outcomes. The final interpretation is what matters

The real value comes from the analysis and interpretation of the clusters — such as:

- **Profiling of each cluster:**
  - Identify dominant characteristics of each cluster
  - Mean, median, mode of each cluster
  - What features differentiate the clusters
- **Assigning meaningful labels** (e.g., "loyal customers", **"tech-**savvy users", "support complaints")

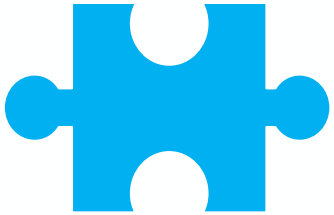
Exercise 4: Online Retail Challenge - Exercise Summary

Objective: This assignment will guide you through customer segmentation using the *Online Retail* dataset. You'll learn how to preprocess data, apply K-Means clustering, and interpret customer segments based on purchasing behavior.

Background: The dataset contains transactional data for a UK-based online retail store from 2010 to 2011. The main goal is to segment customers into distinct groups based on their purchasing behavior using K-Means clustering. This analysis can help businesses identify different types of customers, allowing them to tailor marketing and customer service strategies to specific segments.

No.	Attribute Name	Description	Data type
1	InvoiceNumber	6-digit unique number for each transaction	Nominal
2	StockCode	5-digit unique number for each product	Nominal
3	Description	Product Name	Nominal
4	Quantity	Quantity of product per transactions	Numeric
5	InvoiceDate	Invoice Date and Time	Numeric
6	UnitPrice	Product price per unit	Numeric
7	Customer Id	5-digit unique number for each customer	Nominal
8	Country	Country Name	Nominal

Reference: Chen, D. (2015). Online Retail [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5BW33.



Questions

Answers